

Dataflow vs. an Optimal Heterogeneous Architecture

A process-normalized comparison, the context-switching invariant, the oracular-compiler question, and the processor as contract-keeper

Built from the WaveScalar line (Swanson et al., MICRO 2003 → RTL/ISCA 2006 → thesis/TOCS 2007) plus the fabricated tiled chips (Raw, TRIPS, KiloCore), a modern CGRA (Plasticine), the FPGA → ASIC gap (Kuon & Rose), the heterogeneous von Neumann/dataflow work (Nowatzki & Sankaralingam), and the speculative-parallelism line (Swarm and successors, Sanchez group). Confidence is flagged ●●● measured / ●●○ derived / ●○○ inferred. Numbers across processes carry large error bars — see §3.

1. The right question

Your instinct from message 20 — “you can arrange your cores in a dataflow too with the compiler, so the evidence would have to be in best-case throughput per area or power” — is exactly the correct frame, and it dissolves most of the WaveScalar paper’s framing.

“Dataflow vs. von Neumann” is **not** the real axis, for two reasons:

1. **Any spatial substrate can be compiled into dataflow execution.** A tiled array of plain RISC cores (Raw, KiloCore) with a good mapping is a dataflow machine — instructions placed in space, operands streamed point-to-point. The execution model is a compiler choice, not a hardware property.
2. **A big out-of-order core already executes dataflow.** Register renaming + the issue window dynamically reconstruct the data-dependence graph and fire instructions when operands are ready. OOO is dynamic dataflow with a von Neumann front-end.

So the only thing dedicated dataflow **hardware** can buy you is **efficiency** — making the dependence graph *explicit* so the hardware stops rebuilding it every cycle (saving the energy of rename, CAM-based wakeup, and broadcast bypass), and packing more ALUs per mm². That is an **area/energy** story, not a throughput story. The right metrics are therefore best-case **throughput per mm²** and best-case **throughput per watt** — which is what the tables below try to pin down.

2. The comparison

Table 1a — Physical and best-case performance

Perf/area and energy/op are expressed as **best-case multipliers vs. a general-purpose OOO superscalar (= 1.0)**, on each architecture’s *favorable* workload. “Best case” matters: on a mismatched workload every spatial/dataflow row collapses below 1.0 (that collapse is the whole heterogeneity argument — see §4).

Architecture	Class	Node	Die area (native)	Clock	Best-case throughput (published)	Perf / mm² vs OOO	Energy / op vs OOO
Aggressive OOO superscalar (WaveScalar paper’s baseline)	von Neumann ISA, dynamic dataflow in HW	~90 nm (sim)	n/a (16-wide, 1024-entry window, 1024 phys-regs — idealized, no real chip)	~2 GHz assumed	~1–2 realized IPC typical (16 peak issue)	1.0 (def)	1.0 (def)
Raw (MIT, 2002)	Tiled simple RISC cores, spatial	180 nm	~330 mm² (16 × ~16 mm² tiles, single-issue, 32 KB/tile)	~425 MHz	High on streaming/StreamIt; weak single-thread	~2–4× streaming ●○○	~2–3× streaming ●○○
TRIPS (UT Austin, 2007)	EDGE: static placement, dynamic issue, dataflow firing; still PC-sequenced at block level	130 nm	~336 mm² (2 cores, 1 MB on-chip) ●○○	~366–500 MHz	~within 10% of WaveScalar on shared apps	~1.3–1.5× ST ●●○	~1.5–2× ●○○
WaveScalar / WaveCache (UW, RTL 2006)	Pure tagged-token dynamic dataflow + wave-ordered memory	90 nm	19–378 mm² (rep. 252 mm², 16-cluster)	~1.1 GHz (22 F04)	ST ~2.3 AIPC; multithreaded Splash-2 ~50 IPC; hand-coded dataflow kernels ~120–125 IPC	~1.4× ST; up to ~5–10× MT/kernels ●●○	~1.5–3× (inferred) ●○○
KiloCore (UC Davis, 2016)	1000 simple 16-bit von Neumann cores, spatial MIMD, no caches	32 nm	64 mm² (1000 × 0.055 mm²)	up to 1.78 GHz	peak ~1.78 T-ops/s; ~5.3–5.8 pJ/op	~3.1× vs CPU/GPU on target kernels ●●●	~16.7× vs CPU/GPU ●●●
Plasticine (Stanford, 2017)	CGRA — word-level reconfigurable spatial dataflow	28 nm	113 mm²	1 GHz	12.3 SP TFLOPS, 49 W, 16 MB on-chip → ~109 GFLOPS/mm², ~0.25 TFLOPS/W	very high on parallel patterns; 77× perf/W vs FPGA ●●●	~10–100× vs CPU ●●○
FPGA (reconfigurability reference)	Bit-level reconfigurable	90 nm (Kuon-Rose)	—	—	(fixed-function realized on a flexible fabric)	~0.03–0.05× of ASIC ●●●	~0.07–0.1× of ASIC ●●●
ASIC / fixed-function (efficiency ceiling)	None	—	—	—	function-specific	~100–1000× vs OOO ●●●	~100–500× vs OOO ●●●
OOO2 + SEED (Wisconsin, 2015)	Hybrid: small OOO core + explicit-dataflow engine	sim	< area of an OOO4 core	—	within 15% of a big OOO4 at 2.3× less energy	~1.3–1.7× vs its host ●●●	~1.5–2.3× ●●●
Swarm (MIT, 2015) → Fractal '17, Chronos '20, T4 '22	Many simple cores + HW task scheduling + ordered speculation (TLS/HTM lineage; von Neumann cores)	sim (ZSim; 64–256 cores)	per-core speculation HW overhead modest ●○○	n/a (sim)	51–122× vs 1 core; 2.7–18.2× vs best software-parallel on ordered-irregular (graphs, discrete-event sim, in-memory DB)	unlocks parallelism nothing else can on ordered-irregular; ≈1× elsewhere ●●○	better than software-parallel — less wasted work via priority-ordered commit ●○○

Baseline caveat: rows are measured against different baselines (the paper's OOO for WaveScalar/TRIPS; a real i7/GPU for KiloCore/Plasticine; an ASIC for the FPGA row; the host core for SEED). Treat the multipliers as shape, not as directly subtractable. KiloCore's ops are 16-bit; Plasticine's are FP32 — not the same "op." Swarm's numbers are speedups over a tuned software-parallel baseline on its target irregular workloads, in simulation — a different kind of number again, included because it captures an axis (speculation) the other rows can't.

Table 1b — Operational tradeoffs (where the real differences live)

Architecture	Context-switch / eviction cost	Reconfig granularity	Compiler burden & risk	Best-fit workload	Worst-fit workload
OOO superscalar	Cheap — save PC + regs, tens of cycles	n/a	Mature; scheduling only	Irregular, branchy, multi-tasking	Embarrassingly parallel (wastes area)
Raw	Heavy — tile state + network in flight	Recompile/remap	Spatial mapping; placement-sensitive	Streaming, systolic	Irregular control
TRIPS	Moderate — block state	Recompile	Block formation + placement	High-ILP regions	Tiny basic blocks, hard-to-predict control
WaveScalar	Very heavy — state smeared across the fabric (mapped instrs, input queues, token store); a miss is "heavyweight", termination is a "big hammer", matching table spills to memory	Recompile/remap	NP-hard placement; perf swings with placement quality (TRIPS beat it on some apps purely from bad placement)	Steady-state loops, high MLP/ILP	Frequent context switches, interrupts, short-lived tasks, irregular control
KiloCore	Heavy per-core, but cores are tiny & cheap to flush	Recompile/remap	Partition into ≤ 100 -instr tasks	Fine-grained streaming kernels	Large working sets, single-thread
Plasticine (CGRA)	Config swap = $\sim \mu\text{s}$	Word-level , fast	Pattern mapping (hard by hand; the paper's pitch is automating it)	Nested parallel patterns, dense+sparse	Highly irregular pointer-chasing
FPGA	Full reconfig $\sim \text{ms}$; partial slow	Bit-level , slow	HLS or RTL; long compile	Bit-manipulation, custom datapaths	Anything needing fast phase changes
ASIC	n/a (one function)	None (tape-out)	None at runtime; huge NRE	The one thing it was built for	Everything else
OOO2 + SEED	Cheap (host core handles switches)	Fine-grain OOO+dataflow per phase	Region selection	Mixed code with parallel + irregular phases	— (that's the point)
Swarm (+ Fractal/Chronos/T4)	Moderate — speculative task state + version buffers; OS preemption drains/aborts speculation	n/a (recompile)	Task + timestamp model; scheduling & speculation live in hardware ; T4 auto-extracts tasks from sequential code	Ordered irregular parallelism (graphs, discrete-event sim, transactions, branch-and-bound)	Embarrassingly data-parallel (use CGRA/GPU); genuinely serial code

Swarm and its successors represent a **fourth axis** the other rows miss. Instead of *removing* control to expose parallelism (dataflow/CGRA) or merely replicating simple cores (KiloCore), Swarm *embraces ordering* and uses hardware speculation — running thousands of timestamped tasks ahead of the commit point and rolling back on conflict — to mine parallelism from irregular, ordered code (graph algorithms, discrete-event simulation, transactions, branch-and-bound) that every spatial row above runs terribly. It is the von Neumann lineage's answer to exactly the workloads dataflow can't touch, and it is directly relevant to your second question because *its scheduler is in hardware* — see §5.1.

Table 2 — The reconfigurability tax (the clean, apples-to-apples set)

These are **same-paper**, **same-node** ratios, so they're the most trustworthy numbers here. This table is the empirical backbone of the oracular-compiler answer (§5).

Substrate	Area vs ASIC	Speed vs ASIC	Power vs ASIC	(Re)config time	Source
ASIC (fixed)	1×	1×	1×	n/a (mask)	ceiling
CGRA (word-level, Plasticine)	$\sim 2.8\times$	$\sim 1\text{--}2\times$	$\sim 2\text{--}4\times$	$\sim \mu\text{s}$	Prabhakar et al., ISCA 2017 (2.8× area vs ASIC; 77× perf/W vs FPGA)
FPGA (bit-level)	$\sim 20\text{--}35\times$ ($\approx 18\text{--}21$ with hard DSP/RAM blocks; could approach <5 with heavy hard-block use)	$\sim 3\text{--}4\times$	$\sim 10\text{--}14\times$	$\sim \text{ms}$ (full); partial still slow	Kuon & Rose, TCAD 2007 (90 nm)
GPP / OOO core (fully programmable)	$\sim 50\text{--}500\times^*$	$\sim 1\times$ per op, but low utilization	$\sim 100\text{--}500\times$	$\sim$ tens of cycles	Hameed et al., ISCA 2010 (GP CPU $\sim 500\times$ less energy-efficient than ASIC for H.264)

* per fixed function; the GPP "area vs ASIC" figure is inherently fuzzy.

The price list for flexibility, read off Table 2: going from a fixed ASIC to *word-level* runtime flexibility costs $\sim 2.8\times$ **area**; going all the way to *bit-level* flexibility costs $\sim 20\text{--}35\times$ **area** and $\sim 10\text{--}14\times$ **power**. There is a large, discrete jump between word-level and bit-level reconfigurability — and almost nothing useful in between.

3. Normalization methodology & caveats (so nobody gets an unfair win)

- Area** is scaled between nodes by ideal $(\text{node_target} / \text{node_source})^2$ density. Useful for intuition, but real shrinks never hit ideal density (routing, SRAM, and analog scale worse than logic), so cross-node area comparisons are optimistic for the older, larger-node designs (Raw at 180 nm, TRIPS at 130 nm).

- **Power / energy is the weakest axis.** Dennard scaling broke down around 2005, so normalizing power across 180 nm → 28 nm is not physically meaningful. Energy/op multipliers are therefore reported only where a paper gives real power, and marked inferred otherwise. **No power numbers were fabricated** for WaveScalar, TRIPS, or Raw.
- **Peak vs. sustained.** Using *peak* ALU throughput badly flatters low-utilization dataflow fabrics — WaveScalar's ~512 PEs at 1 GHz imply ~512 "peak G-ops/s," but realized single-thread AIPC is ~2.3. The table therefore uses **sustained best-case** numbers (the workload each design was built for), which is the fair reading of your "best-case throughput per area."
- **Different "ops."** KiloCore is 16-bit integer; Plasticine is FP32; the OOO baseline is 64-bit. A KiloCore "op" and a Plasticine "FLOP" are not commensurable; treat cross-class comparisons as order-of-magnitude.
- **The most reliable single fact in this whole document** is the WaveScalar authors' own mature result, because it controls for compiler and process: a 90 nm WaveCache reaches **single-thread parity with an aggressive OOO superscalar at ~70% of its area** (Swanson thesis / TOCS 2007). That is ~1.4× perf/area on single-thread — **not** the 2–7× headline of the 2003 paper, which was measured against an *idealized simulated* superscalar fed by a *binary translator* with no WaveScalar-aware compiler.

4. Answer 1 — How much benefit is dataflow vs. an optimal heterogeneous machine?

Short version: the dataflow-specific advantage is small, it shrinks as the baseline gets more capable, and against a properly heterogeneous design it is strictly dominated.

Evidence the advantage is small and shrinking:

- **WaveScalar's own numbers.** Mature single-thread result is parity-at-70%-area (~1.4× perf/area), not 2–7×. And in the 2003 paper's own cross-comparison, once TRIPS is given 16-node clusters and full bypassing, WaveScalar's lead **drops to ~11%**. So a pure dataflow machine is ~11% faster than a *hybrid* (TRIPS, which is still PC-sequenced at the block level) once you equalize the spatial/bypass hardware. That ~11% is the price of the program counter, and it is small.
- **SEED is the decisive experiment** (Nowatzki, Gangadhar & Sankaralingam, ISCA 2015 / CACM 2019). They bolt an explicit-dataflow engine onto in-order, OOO2, and OOO4 cores and measure the dataflow contribution directly:
- Speedup **1.65× / 1.33× / 1.14×** over little / medium / big cores.
- The advantage **monotonically shrinks as the core grows**, because a big OOO core is *already doing the dataflow* — explicit dataflow just stops paying to rediscover it.
- Dataflow is profitable on only **71% / 64% / 42%** of execution for the little/medium/big cores — i.e., it's a **regional** win, and per-region it swings violently (3–5× speedups *and* 3–5× slowdowns are both common).
- The energy story is the real one: explicit dataflow is **1.5–1.7× more energy-efficient** because it skips rename/CAM-wakeup/broadcast on the regions where it applies.

Why heterogeneous wins outright: SEED's headline is that **OOO2 + dataflow engine lands within 15% of a big OOO4 while using 2.3× less energy and less area**. The heterogeneous design beats *both* pure approaches — pure OOO (too much energy) and pure dataflow (collapses on irregular code). An "optimal heterogeneous" machine puts a spatial/dataflow fabric on the parallel kernels (where perf/area and perf/W are 3–100×) and a von Neumann core on everything else; pure dataflow is just that machine with the von Neumann core deleted, which is why it can't be general-purpose.

The tradeoffs you predicted — both real, both directions:

- **Compilation.** Spatial/dataflow shifts work onto the compiler: placement is NP-hard and *quality-sensitive*. The WaveScalar paper itself shows TRIPS winning on some apps purely because WaveScalar's greedy placement spread a dependence chain across clusters. Von Neumann needs only instruction scheduling and is insensitive to layout. (This is also exactly why your "put an LLM/oracle in the loop" idea bites hardest *here* — see §5.)
- **Context switching — the killer, and the invariant.** A dataflow machine's live state is *smear*ed across the whole fabric: mapped instructions, per-PE input queues, the token store. There is no compact "context" to save. WaveScalar's own paper calls an eviction "*heavyweight*" and program termination a "*big hammer*," and the matching table spills to DRAM under pressure. So these are **throughput/batch engines**, structurally bad at interrupts, multitasking, short tasks, and OS-style control churn. A von Neumann context switch is save-PC-plus-registers in tens of cycles. **This single asymmetry is why no pure dataflow machine ever became a general-purpose CPU** — and it is the same invariant that governs your second question.
- **Memory ordering.** Dataflow had to *invent* wave-ordered memory to run C at all (sequence numbers, MEMORY-NOPs, store-buffer coordination). It works, but it's overhead that von Neumann gets free from the PC.
- **Control speculation.** WaveScalar's baseline does *none*; perfect branch prediction would add ~47%. Unpredictable branchy code starves the fabric. OOO's whole reason for existing is to eat exactly that code.

5. Answer 2 — With an oracular compiler, what configurability would we choose?

This is the most interesting question, and the answer falls out cleanly once you separate the two things "flexibility" is paying for.

An oracle compiler collapses the *programmability* axis but not the *runtime-adaptability* axis.

The flexibility–efficiency tradeoff is really two independent taxes bundled together:

1. **A programming/compilation tax** — flexible fabrics are hard to target (the Plasticine paper's own motivation: "*CGRAs traditionally require low-level programming and suffer from long compilation times*"); placement quality is a lottery; you risk miscompilation. **An oracle erases all of this.** It gives optimal placement for free, eliminates the placement lottery, and — combined with your message-10 idea of an equivalence check before/after — removes correctness risk. With an oracle, the *usability* objection to CGRAs/FPGAs/spatial fabrics simply vanishes.
2. **A runtime-adaptability tax** — flexible fabrics also cost area/power *at runtime* (Table 2: $\sim 2.8\times$ for word-level, $\sim 20\text{--}35\times$ for bit-level), and they cost *time* to switch configurations. **An oracle erases none of this.** It is a property of the silicon, not the toolchain.

So the configurability you choose is set entirely by item (2), governed by two physical quantities:

- **(a) Workload entropy that survives compile-time prediction.** If your workload set is fixed and known, push everything to fixed-function/ASIC ($1\times$ efficiency); paying the reconfigurability tax on the predictable part is pure waste. You only buy flexibility to cover the part that is *genuinely unknown or changing at runtime*. **The reconfigurability you should purchase equals the residual entropy of your workload after the oracle has predicted everything it can** — and the price list for that entropy is Table 2: $\sim 2.8\times$ area for general word-level flexibility, $\sim 20\text{--}35\times$ for bit-level. Below $\sim 2.8\times$ you can't get word-level generality; above it (FPGA-grain) you're paying $10\times$ the area and power for bit-manipulation you rarely use.
- **(b) Context-switch frequency and granularity** — your "*unavoidable context switching*," and the part the oracle cannot touch. Reconfiguration has latency: a CGRA loads a new config in $\sim \mu\text{s}$; an FPGA full reconfig is $\sim \text{ms}$. If program phases turn over every few μs , FPGA-grain reconfiguration is unusable **no matter how perfect the compiler is** — you physically cannot reload the fabric fast enough. Two consequences:
 - It pushes you toward **coarse, word-level reconfigurability (CGRA) over bit-level (FPGA)** — faster to reconfigure, and far cheaper (Table 2).
 - More importantly, it pushes you toward **a heterogeneous set of resident fixed blocks rather than one maximally-reconfigurable fabric**. If you keep several specialized blocks powered and configured, a phase change is a cheap *dispatch* (route operands to a different block) instead of an expensive *reconfiguration*. You trade area (more resident blocks) for switch latency — and with the dark-silicon reality that you can't power all your transistors anyway (Esmailzadeh et al., ISCA 2011), spending that dark area on a *diversity of seldom-used fixed blocks* is close to free.

So the oracle-optimal machine is NOT "maximally reconfigurable." It is:

fixed-function / ASIC blocks for the stable common case → a modest word-level (CGRA) fabric sized to exactly the compile-time-unpredictable variation → a small von Neumann core for the irregular tail and the context-switch glue.

An oracle compiler **moves the fixed/flexible line further toward fixed** (because the usability cost of fixed blocks — you can't easily reprogram them — is what made flexibility attractive, and the oracle makes targeting fixed blocks easy too). It does **not** eliminate the line.

The deep symmetry between your two questions: *context switching is the invariant in both.*

- In Q1 it's why pure dataflow can't be the whole machine — state is smeared, switching is heavyweight → you need a von Neumann core for the switch-heavy glue.
- In Q2 it's why an oracle-compiled machine still isn't pure-ASIC or pure-fabric — switching configurations is costly → you need resident heterogeneous blocks.

Both converge on the **same structure**: heterogeneous, von Neumann handling the irregular switch-heavy code, specialized/spatial blocks handling steady-state kernels. The oracle changes *where you draw the fixed/flexible line*, not *whether* you draw it — and the thing fixing the line in place, in both cases, is the cost of changing what the hardware is doing.

5.1 — So how far can an oracle compiler and scheduler actually get? (the quantitative ceiling)

Separating the *programmability* tax from the *runtime* tax also answers the quantitative question you flagged. An oracle's payoff decomposes into four pieces — three it can capture, one it can't — and the sizes are measurable:

1. The fundamental ceiling (limit studies). How much parallelism is even *in* the program for a perfect compiler+scheduler to find? The classic oracle studies are unambiguous: under a *single* flow of control with realistic prediction, ILP saturates at $\sim 2\text{--}8$. The dominant barrier is **control flow, not data dependence** — Lam & Wilson (ISCA 1992) showed that removing the control-flow limit (perfect control speculation / following multiple flows at once) raises parallelism by an order of magnitude or more: into the **tens** for irregular integer code and the **hundreds-to-thousands** for regular numeric code. Wall (ASPLOS 1991) found the same shape across his "stupid → perfect" ladder of assumptions. So the oracle's *headroom* is workload-defined — roughly **5–20× on control-bound irregular code, 100–1000× on regular code** — and bounded below by true data-dependence height (the critical path), which no scheduler can shorten.

2. Speculation losses the oracle recovers. Imperfect branch and memory-dependence prediction. The cleanest numbers are in the paper you annotated: a WaveCache with *perfect* branch prediction gains **+47%**, with *perfect* memory disambiguation **+62%**, and with **both, +340% ($\times 4.4$)**. So a perfect predictor/scheduler is worth roughly **2–4.4×** on a dataflow machine — and similarly on an OOO core, whose entire rename/reorder/load-store-queue apparatus exists precisely to approximate this and still falls short.

3. Mapping/scheduling losses the oracle recovers. Heuristic place-and-route and modulo scheduling vs optimal. WaveScalar's own results show TRIPS *beating* it on some apps purely from bad instruction placement; optimal placement recovers that gap, and ILP-based optimal schedulers for spatial fabrics (e.g. Nowatzki's constraint-centric scheduler) show **$\sim 10\text{--}50\%$** over greedy. Real, but second-order.

4. The parallelism-discovery loss — the big, workload-dependent term. This is where an oracle compiler+scheduler earns its keep, and it is exactly what your questions were circling. On *already-parallel* code it buys $\sim 1\times$. On *irregular ordered* code — where the parallelism is genuinely there but hidden behind data-dependent control and ordering constraints — conventional compilers and hardware find almost none of it, and a near-oracle scheduler finds a lot. This is precisely Swarm's measured result: its hardware scheduler turns ≈ 0 **parallel speedup** $\rightarrow 51\text{--}122\times$, and **T4's compiler (ISCA 2022)** auto-extracts the tiny tasks and scales hard-to-parallelize SPEC CPU2006 benchmarks — ones prior work got *no* speedup on — to tens of cores. Swarm + T4 are, in effect, a working approximation of "the oracle scheduler for ordered irregular code," and the gap they close ($1\times \rightarrow 10\text{--}100\times$) is the largest term in the decomposition.

What the oracle cannot touch (the same invariant as Q1): true data-dependence height, genuine serialization (Amdahl), inter-unit communication latency, and reconfiguration/context-switch cost. A long dependent scalar chain runs at the same speed under a perfect compiler as under a dumb one.

Net — a perfect compiler + scheduler is worth, very roughly:

- $\sim 1.5\text{--}4\times$ from recovering scheduling + speculation losses (fairly architecture-independent), *times*
- $\sim 1\times$ to $\sim 10\text{--}100\times$ from parallelism discovery — near-nothing on already-parallel or serial code, up to two orders of magnitude on irregular *ordered* code, **but only on a speculation-capable substrate** (a CGRA still can't run that code: its problem isn't scheduling, it's data-dependent control and conflicts that need rollback), *approaching*
- **the hardware roofline ($\sim 100\text{--}1000\times$ vs a scalar core)** on regular data-parallel code run on a spatial fabric.

The punchline that closes the loop with §6: the oracle does **not** shrink the case for heterogeneity — it *widens* it. A perfect compiler+scheduler lets each substrate reach its own ceiling (CGRA $\rightarrow$ roofline on regular kernels; Swarm-class $\rightarrow$ its $50\text{--}122\times$ on ordered-irregular; OOO $\rightarrow$ its $\sim 4\times$ speculation headroom on the serial tail), but it **cannot make one substrate cover another's regime**, because the regimes differ in *what kind of barrier* limits them — scheduling vs control/conflicts vs raw dependence height — not in how cleverly they're compiled. The oracle makes a heterogeneous machine *more* worth building, because every block finally runs at its roofline instead of its realized average. That gap — roofline minus realized — is exactly the distance between the "best-case" and the everyday columns of Table 1a.

6. What the "optimal heterogeneous architecture" actually is

Putting §4 and §5 together, the target you keep circling (and that your message-16 "heterogeneous von Neumann/dataflow/CGRA/FPGA/ASIC" intuition already sketched) is a **dispatch hierarchy**, ordered by how predictable the code is:

1. **Fixed-function / ASIC blocks** — the stable, high-volume kernels (codecs, crypto, GEMM, FFT). $\sim 100\text{--}500\times$ the energy efficiency of a core. Resident, cheap to dispatch to.
2. **Word-level CGRA fabric** — the parallel kernels that vary at runtime. $\sim 2.8\times$ ASIC area, μs reconfig, excellent perf/W (Plasticine-class). This is the "dataflow hardware" worth building.
3. **A few simple spatial cores (KiloCore-class)** — fine-grained streaming that doesn't fit the CGRA's pattern templates. Tiny (0.055 mm^2), 5 pJ/op, compiled into dataflow pipelines.
4. **One (or few) von Neumann OOO cores** — the irregular, branchy, pointer-chasing tail *and* the orchestration/OS/context-switch glue. The piece every pure-dataflow proposal is missing.
5. **A speculation-capable many-core tier (Swarm-class)** — for *ordered irregular* parallelism (graphs, discrete-event simulation, transactions, branch-and-bound) that is genuinely parallel but invisible to the spatial fabrics. Hardware task scheduling + speculation extract $10\text{--}100\times$ where everything above gets $\approx 1\times$; with T4-style compilation the programmer needn't even mark the tasks.
6. **Bit-level FPGA fabric** — **only** if you genuinely need bit-manipulation or post-silicon datapath changes, because its tax ($\sim 20\text{--}35\times$ area, $\sim 10\text{--}14\times$ power) is rarely worth it once an oracle removes the *programming* excuse for it.

The reason this isn't built today is precisely the two costs the oracle attacks: the **design effort** to build and verify such a beast, and the **compiler** to partition arbitrary programs across six execution styles. Which is exactly the place your "unlimited engineering via machine intelligence" framing (message 16) and your "LLM-in-the-loop compiler with equivalence checking" framing (message 10) point — automated accelerator *synthesis* + automated *mapping* are the two enabling technologies, and both are now active research (see DSAGEN below).

7. The self-optimizing processor: from instruction stream to contract

Everything in §4–§6 is, at bottom, about **widening the set of legal behaviors the hardware is allowed to choose among**, then choosing well. Von Neumann execution honors the program at the tightest possible granularity — every architectural state must match, in order. Each model we examined loosens that:

- **Dataflow / OOO** — the contract is the data-dependence graph plus the final memory state; *any firing order* that respects it is legal. (Relax *order*.)
- **Speculation / Swarm** — the contract is the sequential semantics; any execution *indistinguishable after commit* is legal, even mis-speculations that are rolled back. (Relax *order* *and* provisional wrong states.)
- **Approximate computing** — the contract is a *quality bound on the output*, not bit-exactness; any result within tolerance is legal. (Relax exact *values*.)
- **Algorithm substitution** — the contract is the input $\rightarrow$ output relation (exact via equivalence, or approximate via a quality bound); any implementation satisfying

it is legal. (Relax the *implementation*.)

- **Learned / collaborative optimization** — the *policy* for choosing among legal behaviors improves over time and across machines. (Relax the assumption that the optimization is fixed.)

Read this way, the discussion has really been about **what a processor with inherent intelligence would do** — and the cleanest statement of the principle is yours: such a processor treats the code as a **contract**, the way a person treats one. Not a command with no alternative, but a conditional promise — *"if you do this, the user-visible outcome will be acceptable."* The literal instructions are merely one sufficient implementation of the intent; the intent is the outcome the user actually cares about. The intelligent processor honors the contract only at the **user-visible granularity** — the only granularity the user cares about — and stays free in everything below it.

7.1 — The building blocks already exist (just never unified)

Approximate computing — the output-quality contract. Loop perforation (Sidiroglou-Douskos, Misailovic, Hoffmann & Rinard, FSE 2011) skips work under an accuracy goal; EnerJ (Sampson et al., PLDI 2011) types data as approximate. The feedback-driven systems are closest to a self-optimizing processor: Green (Baek & Chilimbi, PLDI 2010) and PowerDial / "dynamic knobs" + Application Heartbeats (Hoffmann et al., ASPLOS 2011 / ICAC 2010) run a *control loop* with a quality monitor to hold a target as load shifts; SAGE (Samadi et al., MICRO 2013) does runtime-calibrated GPU approximation. Neural acceleration (Esmailzadeh et al., MICRO 2012; SNNAP, HPCA 2015) is the purest "swap the algorithm" case — train a neural surrogate for a code region and offload it. Anytime algorithms (Dean & Boddy, 1988) are the theoretical ancestor.

Algorithmic choice — the implementation contract. PetaBricks (Ansel et al., PLDI 2009) makes algorithmic choice a first-class construct and autotunes whole-algorithm decisions, discovering non-intuitive poly-algorithms; OpenTuner generalized the search; FFTW / ATLAS / SPIRAL do it per kernel; algorithm portfolios (SATzilla) do input-driven selection. Almost all decide at install/compile time, not continuously.

Runtime re-optimization — the online loop. Dynamo (Bala et al., PLDI 2000) re-optimized hot binary traces at runtime and sometimes beat static -O2; Transmeta's Code Morphing did x86 the same way; adaptive JITs (HotSpot, V8) are the successful real version — profile, recompile, deopt, reopt. On the hardware side, the WaveCache's own dynamic instruction placement and Composite Cores (Lukefahr et al., MICRO 2012, sub- μ s engine switching) are incremental hardware optimization.

7.2 — The measurement gap (the prerequisite)

To my knowledge there is **no single study** that traces the whole consumer workload mix and reports max-vs-achieved parallelism *and* dataflow locality together. The pieces live in separate literatures: achieved parallelism on real interactive code (Flautner et al., ASPLOS 2000; Blake, Dreslinski, Mudge & Flautner, ISCA 2010 — both found achieved TLP stuck near ~ 2 even as cores grew); the maximum-parallelism limit studies (Wall; Lam & Wilson — but on benchmarks, not a consumer trace); mobile full-system characterization (Gutierrez et al., IISWC 2011); and *dataflow locality* as a metric, measured essentially only in the WaveScalar paper (its cousin "value locality" is Lipasti, Wilkerson & Shen, ASPLOS 1996). Nobody has unified them on a real phone/laptop workload — and that unification is the prerequisite for the processor below, because **you cannot build the controller without the cost/benefit model, and you cannot get the model without the study**.

7.3 — Code as a contract, not a command

The frame above is unifying, but the genuinely *new* capabilities an intelligent processor would add — beyond every system in §7.1 — are three, and naming them sharpens what "inherent intelligence" has to mean:

1. **Inferring the contract.** Every existing system requires the *programmer* to annotate the contract — EnerJ's qualifiers, PetaBricks's choices, Heartbeats's goals. The binary alone under-determines intent: is this float a final answer or an intermediate feeding a hash? Is this latency on the UI critical path or a background task? Reconstructing intent from implementation is the new capability, and it is why "intelligence" is the right word — it is a semantic-understanding problem, not a scheduling one.
2. **User-relativity of the contract.** The legal set is parameterized by *this user's* utility. Two users running the same binary have different contracts — one tolerates visual approximation, one is energy-bound, one is latency-bound. "Awareness of the user" makes the objective *personalized* rather than universal, which is a real departure: every system in §7.1 optimizes a fixed objective.
3. **Growing, collaborative knowledge — and the reason the federation actually forms.** The library of optimizations and algorithms, and the policy for choosing among legal behaviors, improve across the population and over time — *"for code shaped like this, run by users like that, this substitution preserves the contract and improves the metric."* That is profile-guided recompilation lifted from one machine to a federation. The hard question is *why* the federation would form, since today optimizations stay siloed: individual software vendors face a **collective-action problem** — a pooled optimization is a non-rival public good no single vendor is incentivized to provide, so each re-derives or hoards it. **Moving the optimization step into the hardware layer (or the runtime the hardware vendor ships) routes around the collective-action problem, because the chipmaker is the natural aggregation point.** It is the *common substrate* (it sees the whole workload distribution — the only vantage from which population-level optimization is even possible); it is *concentrated* (a few firms, so coordination is tractable rather than requiring millions of developers to agree); it already owns a *distribution channel* into the optimization layer (microcode, drivers, firmware, on-device runtimes); and it *captures the aggregate benefit* (better real-world performance on its silicon → competitive advantage → more silicon sold). That triad — vantage, channel, incentive — is exactly what an individual software vendor lacks. This is already real in fragments: GPU vendors ship per-application optimization profiles, aggregated across all users of a title, via driver updates (population-pooled algorithm substitution performed by *the hardware vendor*); mobile runtimes ship cloud profiles so a fresh install is pre-optimized from the population's hot paths; and a vendor that owns silicon + OS + compiler + frameworks (Apple) is the existence proof of the whole stack. The generalization is to make **contract-conditional, population-pooled optimization the explicit**

job of the hardware/runtime layer.

This vision only stays honest if it keeps faith with the invariants the rest of the report established:

- **Relaxation is safe only if the contract is checkable or bounded.** Infer it wrong and you get *silent corruption* — you "optimized" something the user did care about. So the verifier / quality monitor (your equivalence-check idea; approximate computing's QoS controllers) is not decoration — it is what makes the whole scheme safe. **Inference proposes; verification disposes.**
- **The intelligence is itself overhead** — the same amortization invariant. Inferring intent, searching the legal set, and swapping all cost. It pays only where the contract is honored over a long run (the hot, stable regions), and should leave the cold, ever-changing tail alone — which the limit studies say has little headroom anyway.
- **The ceiling is still the oracle ceiling.** Even perfect contract-inference cannot beat data-dependence height or genuine serialization. Its leverage is widest exactly where the legal set is widest — regular, approximable, algorithm-substitutable code — and narrowest on irregular exact code. The frame does not repeal the physics; it lets you exploit all the legal slack the physics leaves.
- **Whoever runs the loop owns the objective — and that may not be the user.** Solving the collective-action problem by *concentrating* optimization in the hardware vendor trades it for a market-power and principal-agent problem. The aggregator optimizes *its* metric — benchmark numbers, battery-life marketing, telemetry, even planned-obsolescence-adjacent throttling (cf. the device-throttling episodes) — which §7.3's user-relativity says should be the *user's* contract. The pooled commons is also walled per vendor (competitors won't share with each other), so the outcome is a few optimization silos and a data-network-effect flywheel that entrenches the incumbent, not one open commons. The central governance question — *who verifies that the pooled optimization served the user's contract rather than the vendor's?* — extends "inference proposes, verification disposes" to include **the user verifying the aggregator.**

So "a processor with inherent intelligence" is not a *different* machine from the heterogeneous dispatch hierarchy of §6 — it is that machine plus a controller whose job is to (a) infer each region's true contract, (b) pick the widest-slack legal behavior that satisfies it *for this user*, (c) verify or bound the result, and (d) share what it learns. §4–§6 answer *which legal behaviors can be executed efficiently* — the mechanism. The contract frame asks *which legal behavior should be chosen, and who decides what "legal" means* — the policy. They are the two halves of the same machine, and the measurement gap of §7.2 is precisely what currently blocks the policy half.

8. Reading list

Closest to your desired table (from the original authors):

- Swanson, Putnam, Mercaldi, et al., *Area-Performance Trade-offs in Tiled Dataflow Architectures*, ISCA 2006 — the WaveScalar Pareto frontier (200+ designs, 19–378 mm², 22 FO4); has the perf-per-area comparisons to OOO and Niagara you're looking for.
- Swanson et al., *The WaveScalar Architecture*, ACM TOCS 25(2), 2007 — the mature journal version with the honest ~70%-area / single-thread-parity result, multithreading, and the "objections to dataflow" section.

The direct answer to your heterogeneity question:

- Nowatzki, Gangadhar & Sankaralingam, *Exploring the Potential of Heterogeneous Von Neumann/Dataflow Execution Models*, ISCA 2015 — **read this first.** The 1.14–1.67× / 1.5–1.7× and the OOO2+SEED-within-15%-of-OOO4 results.
- Nowatzki, Gangadhar & Sankaralingam, *Heterogeneous Von Neumann/Dataflow Microprocessors*, CACM 62(6), 2019 — the accessible writeup.
- Govindaraju et al., *DySER: Unifying Functionality and Parallelism Specialization*, IEEE Micro 2012 (and HPCA 2011) — a CGRA dropped into an OOO pipeline; 2.1× speedup + 40% energy from two blocks; within ~5% of per-kernel dedicated hardware. The concrete "small dataflow block in a von Neumann core" design.

The speculation axis — mining parallelism from *ordered, irregular* code (the new table rows):

- Jeffrey, Subramanian, Yan, Emer & Sanchez, *A Scalable Architecture for Ordered Parallelism (Swarm)*, MICRO 2015 — the 51–122× / 2.7–18.2× results; the canonical entry point.
- Jeffrey et al., *Data-Centric Execution of Speculative Parallel Programs (Spatial Hints)*, MICRO 2016 — scales Swarm to 256 cores (near-linear on 9 apps; up to 16× over hint-oblivious scheduling).
- Subramanian et al., *Fractal: An Execution Model for Fine-Grain Nested Speculative Parallelism*, ISCA 2017 — nested speculation; up to 88× over prior speculative architectures at 256 cores.
- Abeydeera & Sanchez, *Chronos: Efficient Speculative Parallelism for Accelerators*, ASPLOS 2020 — the FPGA-accelerator branch (the SLOT model needs no cache coherence); 3.5–15.3× over software on a pricier multicore. "Make Swarm a spatial accelerator."
- Ying, Jeffrey & Sanchez, *T4 (Compiling sequential code for effective speculative parallelization)*, ISCA 2022 — the *compiler* that auto-extracts tiny tasks and parallelizes SPEC CPU2006 codes prior work couldn't. The most direct evidence for the §5.1 "oracle scheduler on irregular code" argument.

The reconfigurability-tax question (your message 24 — "what tax is allowable if programming cost were zero"):

- Kuon & Rose, *Measuring the Gap Between FPGAs and ASICs*, IEEE TCAD 2007 — the canonical tax measurement (Table 2 here).
- Hameed et al., *Understanding Sources of Inefficiency in General-Purpose Chips*, ISCA 2010 — decomposes *where* the ~500× CPU → ASIC overhead goes

(fetch/decode, register file, control, data supply); shows a heavily customized CMP reaching within $\sim 3\times$ of ASIC energy. This is the quantitative anatomy of "what you pay for generality."

- André DeHon, *Fundamental Underpinnings of Reconfigurable Computing Architectures*, Proc. IEEE 2015 (and his area-time tradeoff work) — the most principled treatment of the flexibility/efficiency Pareto frontier; the closest thing to a theory of "how much reconfigurability is worth it."
- Prabhakar et al., *Plasticine: A Reconfigurable Architecture for Parallel Patterns*, ISCA 2017 — the modern CGRA; the $2.8\times$ -area-vs-ASIC and $77\times$ -perf/W-vs-FPGA numbers.
- Venkatesh et al., *Conservation Cores: Reducing the Energy of Mature Computations*, ASPLOS 2010, and *QsCores* (MICRO 2011) — the "freeze mature code into fixed-function blocks" position; directly informs the §5 "how much should be fixed" answer.
- Esmailzadeh et al., *Dark Silicon and the End of Multicore Scaling*, ISCA 2011 — why you *can't* power all transistors, which is what makes "many resident specialized blocks" nearly free and motivates the §5 conclusion.

The oracle-compiler frontier (your message-10/11 idea, now real research):

- Weng et al., *DSAGEN: Synthesizing Programmable Spatial Accelerators*, ISCA 2020 (Nowatzki group) — automated synthesis *and* compilation of spatial accelerators; the "unlimited engineering" direction made concrete.
- Schkufza, Sharma & Aiken, *STOKE: Stochastic Superoptimization*, ASPLOS 2013 — search-based optimal code generation **with equivalence checking** — exactly your "optimize, then verify equivalence" loop.
- Mankowitz et al., *AlphaDev / Faster sorting algorithms discovered using deep RL*, Nature 2023 — RL-discovered, formally-verified assembly routines; a working instance of "search-based oracle compiler with a correctness guarantee."
- Cummins et al., *Large Language Models for Compiler Optimization*, 2024 (Meta) — LLM-driven optimization passes; the literal version of your message-10 proposal.
- Lopes et al., *Alive2: Bounded Translation Validation for LLVM*, PLDI 2021 — practical equivalence checking for a real compiler IR; the verification backstop that makes an aggressive/learned optimizer safe.

The self-optimizing processor — approximate computing, algorithmic choice, online re-optimization (§7):

- Mittal, *A Survey of Techniques for Approximate Computing*, ACM Computing Surveys 2016 — the landscape; covers finding approximable regions and monitoring output quality.
- Sidiroglou-Douskos, Misailovic, Hoffmann & Rinard, *Managing Performance vs. Accuracy Trade-offs with Loop Perforation*, FSE 2011 — the canonical "skip work under an accuracy goal."
- Baek & Chilimbi, *Green: Energy-Conscious Programming Using Controlled Approximation*, PLDI 2010, and Hoffmann et al., *PowerDial / Dynamic Knobs* (ASPLOS 2011) + *Application Heartbeats* (ICAC 2010) — runtime QoS control loops; the closest existing thing to an incremental-optimizing processor.
- Esmailzadeh, Sampson, Ceze & Burger, *Neural Acceleration for General-Purpose Approximate Programs*, MICRO 2012, and Moreau et al., *SNNAP*, HPCA 2015 — substitute a learned neural surrogate for a code region (the "swap the algorithm" case).
- Samadi et al., *SAGE: Self-Tuning Approximation for Graphics Engines*, MICRO 2013 — runtime-calibrated approximation ($\sim 2.5\times$ at $<10\%$ quality loss).
- Ansel et al., *PetaBricks: A Language and Compiler for Algorithmic Choice*, PLDI 2009, and *OpenTuner*, PACT 2014 — algorithmic choice as a first-class construct; autotuned poly-algorithms.
- Bala, Duesterwald & Banerjia, *Dynamo: A Transparent Dynamic Optimization System*, PLDI 2000 — runtime binary re-optimization that sometimes beat static -O2 (Transmeta's Code Morphing is the commercial cousin); Lukefahr et al., *Composite Cores*, MICRO 2012 — sub- μ s hardware engine switching.

Whole-system & limit characterization — the §7.2 "measurement gap":

- Flautner, Uhlig, Reinhardt & Mudge, *Thread-Level Parallelism and Interactive Performance of Desktop Applications*, ASPLOS 2000 — early measurement: real interactive apps run $\sim 1\text{--}2$ active threads.
- Blake, Dreslinski, Mudge & Flautner, *Evolution of Thread-Level Parallelism in Desktop Applications*, ISCA 2010 — a decade on, achieved TLP on real Windows apps still ≈ 2 despite more cores.
- Gutierrez et al., *Full-System Analysis and Characterization of Interactive Smartphone Applications*, IISWC 2011 — the mobile-side workload characterization.
- Lipasti, Wilkerson & Shen, *Value Locality and Load Value Prediction*, ASPLOS 1996 — the operand-predictability cousin of WaveScalar's "dataflow locality," which remains about the only place that exact metric was measured.

Background on the limits being chased (your messages 9 & 18):

- Agarwal, Hrishikesh, Keckler & Burger, *Clock Rate vs. IPC: The End of the Road for Conventional Microarchitectures*, ISCA 2000 — the wire-delay/complexity wall that motivated all of the above.
- Sankaralingam et al., *Exploiting ILP, TLP, and DLP with the Polymorphous TRIPS Architecture*, ISCA 2003, and *Distributed Microarchitectural Protocols in the TRIPS Prototype Processor*, MICRO 2006 — the fabricated EDGE chip in detail.
- Bohnenstiehl et al., *KiloCore: A 32-nm 1000-Processor Computational Array*, IEEE JSSC 52(4), 2017 — the "many simple cores" datapoint and its perf/area & energy methodology.
- Lam & Wilson, *Limits of Control Flow on Parallelism*, ISCA 1992 — the key result that **control flow, not data dependence, caps ILP**; removing it unlocks

10–100×. The quantitative backbone of §5.1.

- Wall, *Limits of Instruction-Level Parallelism*, ASPLOS 1991 — the "stupid → perfect" ladder of oracle assumptions and the ILP ranges each unlocks; the original oracle study.